



**NIGHTMARE ACADEMY – AN INTERACTIVE
HORROR-BASED WEB QUIZ GAME**



A PROJECT REPORT

Submitted by

THARUNIYA T	2303811710422171
THASWIN BANU M.A	2303811710422172
THREVENI S.A	2303811710422173
YOGA	2303811710422186

in partial fulfillment for the requirements for the award degree of

Bachelor of Engineering

CSB1332 – DESIGN PROJECT

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)**

SAMAYAPURAM-621112

MAY 2026



**NIGHTMARE ACADEMY – AN INTERACTIVE
HORROR-BASED WEB QUIZ GAME**



A PROJECT REPORT

Submitted by

THARUNIYA T	2303811710422171
THASWIN BANU M.A	2303811710422172
THREVENI S.A	2303811710422173
YOGA	2303811710422186

in partial fulfillment for the requirements for the award degree of

Bachelor of Engineering

CSB1332 – DESIGN PROJECT

**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**

**K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY
(AUTONOMOUS)**

SAMAYAPURAM-621112

MAY 2026

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY**(AUTONOMOUS)****SAMAYAPURAM-621112****BONAFIDE CERTIFICATE**

The work embodied in the present project report entitled “**NIGHTMARE ACADEMY – AN INTERACTIVE HORROR-BASED WEB QUIZ GAME**” has been carried out by the students **THARUNIYA T, THASWIN BANU M.A, THREVENI S.A , YOGA S** , The work reported herein is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce:

Mrs.R.DEEPA M.E.,(Ph.D.,)**SUPERVISOR**

Assistant Professor,

Department of CSE,

K.Ramakrishnan College of Technology

(Autonomous)

Samayapuram-621112

Mr. R. RAJAVARMAN M.E.,(Ph.D.,)**HEAD OF THE DEPARTMENT**

Assistant Professor(Sr.Grade),

Department of CSE,

K.Ramakrishnan College of Technology

(Autonomous)

Samayapuram-621112

INTERNAL EXAMINER**EXTERNAL EXAMINER**

ABSTRACT

The proposed system is a client-side, web-based riddle and quiz application designed to enhance user engagement and learning through gamification. Developed using HTML, CSS, and JavaScript, it eliminates the need for a backend server, making it a lightweight and highly accessible tool. The application features a horror-themed storyline where users progress through levels by solving multiple-choice questions and image-based puzzles across technical domains such as Artificial Intelligence, Databases, and Operating Systems. A key innovation is its immersive feedback mechanism, which uses audio sound effects and horror-themed voice narration to explain correct answers when a user fails. The system aims to overcome the monotony of traditional static quiz platforms by providing a memorable, sensory-rich environment that promotes independent learning and cognitive skill development.

Keywords: Modular Design, Event-driven Programming, Responsive Design, Gamification, Level-based Progression, Immersive Experience, Immediate Feedback.

ACKNOWLEDGEMENT

We thank our **Dr. N. Vasudevan**, Principal, for his valuable suggestions and support during the course of my research work.

We thank **Mr. R. Rajavarman**, Head of the Department, Assistant Professor (Sr. Grade), Department of Computer Science and Engineering, for his valuable suggestions and support during the course of my research work.

We wish to record my deep sense of gratitude and profound thanks to my Guide **Mrs.R.Deepa**, Assistant Professor, Department of Computer Science and Engineering for her keen interest, inspiring guidance, constant encouragement with my work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Mrs.S.Gayathri**, Assistant Professor, Department of Computer Science and Engineering, for her valuable suggestions and support during the course of my research work.

We also thank the faculty and non-teaching staff members of the Department of CSE, K.Ramakrishnan College of Technology (Autonomous), Trichy, for their valuable support throughout the course of my research work.

Finally, we thank our parents, friends and our well wishes for their kind support.

SIGNATURE

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iii
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	INTRODUCTION	1
	1.1 DESCRIPTION	1
	1.2 APPLICATION DOMAIN	2
	1.3 NEED FOR SYSTEM	3
	1.4 PROPOSED SYSTEM	4
	1.5 SCOPE OF THE PROJECT	5
2	LITERATURE SURVEY	6
	2.1 GAMIFICATION-BASE LEARNING SYSTEMS	6
	2.2 WEB-BASED QUIZ APPLICATION DEVELOPMENT	6
	2.3 USER INTERFACE AND USER EXPERIENCE DESIGN	6
	2.4 MODULAR PROGRAMMING IN WEB APPLICATIONS	6
	2.5 CLIENT-SIDE PROCESSING IN WEB APPLICATIONS	7
	2.6 CHALLENGES AND RESEARCH GAPS IN EXISTING SYSTEMS	7
3	EXISTING SYSTEM	8
	3.1 EXISTING SYSTEM	8
	3.2 DISADVANTAGES	9
4	PROBLEMS IDENTIFIED	10
5	PROPOSED SYSTEM	11

	5.1 PROPOSED SYSTEM OVERVIEW	11
	5.2 PROPOSED SYSTEM ARCHITECTURE	13
	5.3 ADVANTAGES	14
6	SYSTEM REQUIREMENTS	15
	6.1 HARDWARE REQUIREMENTS	15
	6.2 SOFTWARE REQUIREMENTS	16
7	SYSTEM IMPLEMENTATIONS	17
	7.1 LIST OF MODULES	17
	7.2 MODULE DESCRIPTION	18
	7.2.1 USER INTERFACE MODULE	18
	7.2.2 QUIZ MODULE	18
	7.2.3 PUZZLE MODULE	19
	7.2.4 GAME LOGIC MODULE	19
	7.2.5 NAVIGATION MODULE	19
	7.2.6 AUDIO & VOICE FEEDBACK MODULE	19
8	SYSTEM TESTING	20
	8.1 UNIT TESTING	20
	8.2 INTEGRATION TESTING	20
	8.3 SYSTEM TESTING	20
	8.4 PERFORMANCE TESTING	21
	8.5 USABILITY TESTING	21
9	RESULT AND DISCUSSION	2
10	CONCLUSION AND FUTURE ENHANCEMENTS	24
	10.1 CONCLUSION	24
	10.2 FUTURE ENHANCEMENTS	25
	APPENDIX A – SOURCE CODE	26
	APPENDIX B – SCREENSHOTS	49
	REFERENCES	52

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	Existing System	9
5.5	Proposed System	13

LIST OF ABBREVIATIONS

AI	- Artificial Intelligence
HTML	- Hyper Text Markup Language
API	- Application Programming Interface
Gui	- Graphical User Interface
HUD	- Heads-Up Display
SQL	- Structured Query Language
CPU	- Central Processing Unit
ML	- Machine Learning
NLP	- Natural Language Processing
RAM	- Random Access Memory
SSD	- Solid State Drive
HTTP	- Hypertext Transfer Protocol
JSON	- JavaScript Object Notation
REST	- Representational State Transfer
JPEG	- Joint Photographic Experts Group
IDE	- Integrated Development Environment
GPU	- Graphics Processing Unit

CHAPTER 1

INTRODUCTION

1.1 DESCRIPTION

The proposed system is a web-based Riddle Game Application designed to provide an interactive and engaging platform for users to test their knowledge and logical thinking skills. The application presents users with a series of riddles and subject-based questions, allowing them to select answers and receive immediate feedback. It is developed using standard web technologies such as HTML, CSS, and JavaScript, ensuring ease of access through any modern web browser without the need for installation.

The system is structured to include multiple categories of questions, including topics such as Artificial Intelligence, Databases, Operating Systems, Computer Networks, and Web Technologies. Each category contains a set of predefined questions stored in separate JavaScript files, which are dynamically loaded and displayed to the user during gameplay. Our modular design allows easy maintenance and scalability of the system.

The user interface is designed to be simple, intuitive, and visually appealing. It incorporates multimedia elements such as images, background audio, and video to enhance user engagement and create an immersive gaming experience. Navigation within the application is smooth, allowing users to move between questions seamlessly.

The core functionality of the system is handled through JavaScript, which manages the logic for displaying questions, validating answers, and progressing through different levels or categories. Event-driven programming techniques are used to handle user interactions, ensuring real-time response and feedback.

1.2 APPLICATION DOMAIN

The proposed system belongs to the domain of interactive web-based gaming and educational applications. It combines elements of entertainment and learning by presenting users with riddles and subject-based questions through a digital platform. Such applications are increasingly popular as they provide an engaging way to test knowledge while improving logical thinking and problem-solving skills.

Our system specifically falls under the category of browser-based applications, as it is developed using HTML, CSS, and JavaScript. It operates entirely on the client side, making it lightweight and easily accessible without the need for complex installations. The inclusion of multiple domains such as Artificial Intelligence, Databases, Operating Systems, Networks, and Web Technologies makes the application suitable for students and learners in technical fields.

The system includes multiple subject domains such as Web Technologies, Database Management Systems, Computer Networks, Operating Systems, and Artificial Intelligence, making it useful for engineering students and technical learners. Before answering quiz questions, users may need to solve small puzzles that improve concentration and logical thinking. The game also provides different difficulty levels such as easy, medium, and hard, allowing students to progress based on their knowledge and confidence. Our structure helps learners improve step by step while maintaining excitement throughout the process.

The horror theme of the project creates a unique atmosphere using dark visual design, background videos, sound effects, suspenseful music, and animated ghost guides. These multimedia elements increase emotional engagement and help users stay focused for longer periods. Since the system is web-based and developed using HTML, CSS, JavaScript, and backend support, it is easy to access and use on computers without complex installation. Overall, Nightmare Academy transforms a normal educational quiz into a memorable gaming experience that supports effective learning and better academic performance.

1.3 NEED FOR SYSTEM

Traditional learning methods often lack engagement and interactivity, which can make it difficult for students to stay motivated. Similarly, conventional quiz systems are often static, offering limited variation and lacking visual or interactive appeal. This creates a need for a system that combines learning with entertainment.

The proposed riddle game system addresses this gap by providing an engaging platform where users can test their knowledge in a fun and interactive manner. It reduces the monotony of traditional study methods by introducing puzzles, images, and multimedia elements such as background music and animations. Another important need for this system is accessibility. Many existing applications require installation or internet dependency for backend processing. Our system, being lightweight and browser-based, ensures easy access and quick usability.

Our system addresses that need by introducing gamification, puzzles, levels, rewards, and a horror-themed environment that makes studying more interactive and memorable. It is especially useful for engineering students who need regular practice in technical subjects such as Web Technologies, DBMS, Computer Networks, Operating Systems, and Artificial Intelligence. The system also helps improve logical thinking, problem-solving ability, and self-assessment through puzzle-solving and adaptive difficulty levels. Since students today are more attracted to visually rich and game-like digital experiences, this project provides a modern solution that supports effective learning while reducing academic pressure and increasing student participation.

Furthermore, there is a growing demand for self-assessment tools among students. Our system allows users to evaluate their understanding across different subjects without external supervision. It promotes independent learning and enhances cognitive skills such as reasoning, memory, and analytical thinking.

This project fulfills that need by introducing gamification features such as puzzles, levels, rewards, and a horror-themed environment that creates excitement and curiosity. The inclusion of sound effects, ghost characters, dark visuals, and storytelling makes the platform unique and helps improve concentration and memory retention.

1.4 PROPOSED SYSTEM

The proposed system follows a modular and client-side web development approach. It is designed using HTML for structure, CSS for styling, and JavaScript for functionality. The application is organized into multiple components to ensure clarity and maintainability. The user interface is developed in a way that is visually appealing and easy to navigate. Background media such as images, audio, and video are integrated to enhance user engagement. Questions are categorized into different domains such as AI, DBMS, OS, Networks, and Web Technologies. These are stored in separate JavaScript data files, making it easy to update or expand the question database without affecting the core system. The system operates entirely on the client side, eliminating the need for server-side processing.

Overall, the approach focuses on simplicity, interactivity, and scalability, allowing the system to be easily enhanced in the future. The proposed system “Nightmare Academy – Horror Quiz Game” is a web-based educational platform developed to make learning more interactive, engaging, and enjoyable for students. The main objective is to reduce boredom in studying and increase student motivation by transforming normal quizzes into an entertaining experience filled with challenges, puzzles, and visual storytelling.

The system is designed for engineering students and covers multiple technical subjects such as Web Technologies, Database Management Systems, Computer Networks, Operating Systems, and Artificial Intelligence. Users can select their preferred subject and begin the game by entering different levels of academic challenges. This multi-domain approach makes the platform more useful for both classroom learning and self-practice, helping students prepare for exams and improve subject understanding.

One of the key features of the proposed system is puzzle-based learning. Before answering quiz questions, users must solve small puzzles or challenges that improve concentration and logical thinking. This creates mental readiness and keeps students actively involved in the learning process. The system also includes multiple difficulty levels such as easy, medium, and hard, allowing users to progress based on their knowledge and confidence level.

1.5 APPLICATIONS OF THE SYSTEM

The riddle game system has multiple applications across different domains. In the field of education, it can be used as a learning tool for students to revise concepts in an interactive manner. It is particularly useful for computer science students, as it includes domain-specific questions. The system can also be used in training environments where quizzes and assessments are required.

It can be used in colleges and educational institutions for conducting quiz-based learning activities, self-assessment tests, and revision sessions in a more engaging way. Engineering students can use the system to practice important technical subjects such as Web Technologies, Database Management Systems, Computer Networks, Operating Systems, and Artificial Intelligence without feeling the boredom of traditional study methods. Teachers can also use this platform during classroom activities to make learning sessions more interesting and improve student participation.

The puzzle-solving feature strengthens logical thinking and problem-solving skills, while multiple difficulty levels support learners at different knowledge stages. Since the system is web-based, it can be easily accessed through computers and laptops, making it suitable for both classroom learning and independent practice at home.

In addition, the application serves as a recreational tool for users who enjoy puzzles and brain games. It helps improve cognitive abilities such as logical reasoning, memory retention, and problem-solving skills. The system can also be adapted for competitive events, online quizzes, or technical competitions. Its modular design allows it to be extended with features such as scoring systems, timers, and leaderboards.

CHAPTER 2

LITERATURE SURVEY

2.1 GAMIFICATION-BASED LEARNING SYSTEMS

Several researchers have explored the impact of gamification in educational applications. Karl M. Kapp (2012) highlighted that integrating game elements such as points, badges, and leaderboards significantly improves learner engagement and motivation. Similarly, Sebastian Deterding et al. (2011) defined gamification as the use of game design elements in non-game contexts and demonstrated its effectiveness in enhancing user participation. These studies prove that quiz and riddle-based systems can make learning more interactive and enjoyable.

2.2 WEB-BASED QUIZ APPLICATION DEVELOPMENT

Research on web technologies shows that modern applications rely heavily on HTML, CSS, and JavaScript for efficient development. Steven Holzner (2004) emphasized that JavaScript enables dynamic content and real-time interaction in web applications. Additionally, Jon Duckett (2011) explained how combining HTML and CSS helps create visually appealing and structured interfaces. These technologies are widely used in quiz systems to ensure responsiveness, cross-platform compatibility, and ease of use.

2.3 USER INTERFACE AND USER EXPERIENCE DESIGN

User interface design plays a crucial role in the effectiveness of interactive systems. Don Norman (2013) discussed that a well-designed interface improves usability and user satisfaction by making systems intuitive and easy to navigate. Research also indicates that the use of multimedia elements such as images, animations, and audio enhances user engagement and retention. These principles are essential in designing quiz and riddle platforms that attract and retain users.

2.4 MODULAR PROGRAMMING IN WEB APPLICATIONS

Modular programming has been widely recommended for building scalable and maintainable systems. David Parnas (1972) introduced the concept of modular design, emphasizing the importance of dividing systems into independent components. This approach simplifies debugging, testing, and future updates. In quiz applications, modules can be developed separately for question handling, scoring, and user interaction, improving system efficiency and organization.

2.5 CLIENT-SIDE PROCESSING IN WEB APPLICATIONS

Client-side processing has become an important area of research in modern web application development. Nicholas C. Zakas (2012) explained that client-side technologies like JavaScript improve application performance by reducing server load and enabling faster response times. Similarly, Stoyan Stefanov (2010) highlighted that browser-based applications provide better scalability and responsiveness, especially for interactive systems like quizzes and games. By handling operations such as validation, scoring, and dynamic content updates on the client side, these systems become more efficient and user-friendly. This approach also enhances accessibility, as users can interact with applications without constant server communication.

2.6 CHALLENGES AND RESEARCH GAPS IN EXISTING SYSTEMS

Despite significant advancements, several challenges still exist in current quiz and riddle systems. John Hattie (2009) emphasized that personalized learning is essential for improving student performance, yet many systems fail to implement adaptive features. Most existing applications provide static question sets without adjusting difficulty based on user performance. Additionally, there is limited focus on data tracking and analytics, which are crucial for monitoring progress and improving learning outcomes. Another major gap is the lack of intelligent recommendation systems that can guide users based on their strengths and weaknesses. These limitations indicate the need for more advanced, adaptive, and data-driven quiz systems that can provide a personalized learning experience.

CHAPTER 3

EXISTING SYSTEM

3.1 EXISTING SYSTEM

The existing systems for quizzes and riddles are often limited in functionality and user engagement. Many traditional systems are static, presenting questions in a simple text format without any interactive or multimedia features. This reduces user interest and makes the experience less engaging. In some cases, existing systems rely heavily on backend processing, requiring internet connectivity and server resources. This increases complexity and may lead to performance issues. Additionally, such systems may not be easily accessible across all devices.

Another limitation is the lack of categorization in many quiz applications. Questions are often presented randomly without grouping based on subjects or difficulty levels. This makes it difficult for users to focus on specific areas of interest or learning. User interfaces in existing systems are often outdated and not visually appealing. The absence of animations, images, or sound effects results in a monotonous experience.

Furthermore, many systems do not provide immediate feedback, which is essential for effective learning. Scalability is also a concern in traditional systems. Adding new questions or features may require significant modifications, making maintenance difficult. In contrast, the proposed system overcomes these limitations by offering a dynamic, multimedia-rich, and categorized riddle platform. Most of these systems are simple, text-based, and designed only for academic assessment without providing interactive or entertaining features. They are commonly developed using basic web technologies and are used in colleges and schools for conducting online tests, assignments, and self-evaluation.

Although these systems are useful for checking knowledge and saving time in manual correction, they often lack creativity, visual appeal, and student engagement, making the learning process feel repetitive and less interesting. It provides a more engaging and efficient solution compared to existing systems.

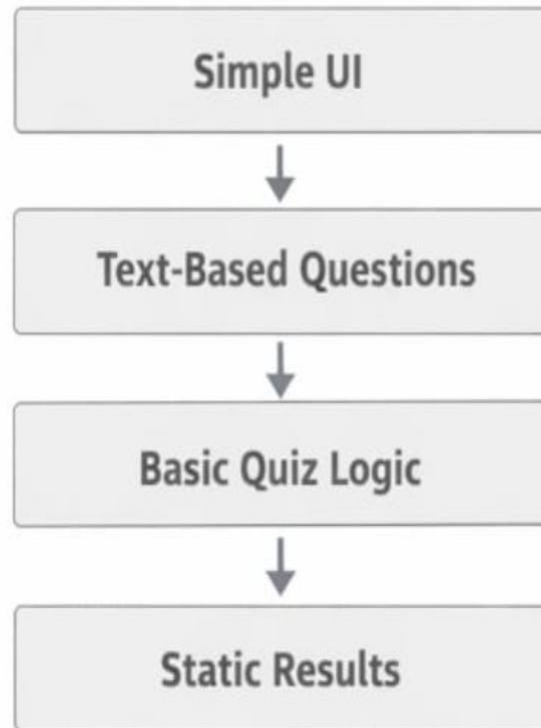


Figure 3.1 Existing System

3.1 DISADVANTAGES

- Mostly static and text-based with no interactive elements
- Low user engagement due to lack of gamification
- No storytelling or thematic learning experience
- Absence of audio effects or voice-based feedback
- Limited user motivation to continue playing or learning
- Repetitive question-answer format becomes boring over time
- No real-time feedback or explanation for wrong answers
- No level-based progression or challenge system
- Does not provide an immersive or memorable learning experience

CHAPTER 4

PROBLEMS IDENTIFIED

Traditional quiz systems are mostly static and lack interactivity, which reduces user engagement significantly. They typically follow a simple question-and-answer format without incorporating any dynamic elements such as animations, storytelling, or challenges. As a result, users often lose interest quickly, making the learning experience less effective and monotonous.

Another major issue is the absence of immersive features such as themes, sound effects, or voice-based feedback. Traditional systems do not provide any emotional or sensory engagement, which is essential for maintaining user attention. Without these elements, the system fails to create a memorable learning environment, limiting its overall impact on users.

Furthermore, existing systems do not provide proper feedback mechanisms. When users answer incorrectly, they are often not given detailed explanations or guidance to understand their mistakes. This lack of instant and meaningful feedback reduces the opportunity for learning and improvement, making the system less educational. Another important problem in existing systems is the lack of personalization and adaptability. Most quiz platforms present the same set of questions to all users without considering their skill level or learning pace. This makes the experience either too easy or too difficult for some users, reducing its effectiveness. Additionally, there is no tracking of user progress or performance over time, which limits the ability to provide a customized and improved learning journey.

Finally, most traditional quiz platforms have outdated and unattractive user interfaces with minimal navigation features. They lack level-based progression, rewards, and challenge mechanisms that could motivate users to continue. This results in a poor user experience and fails to meet the expectations of modern interactive applications.

CHAPTER 5

PROPOSED SYSTEM

5.1 PROPOSED SYSTEM OVERVIEW

The proposed system, Nightmare Academy, is an interactive web-based quiz application designed to transform traditional learning into an engaging and immersive experience. Unlike conventional quiz systems, this platform integrates a horror-themed storyline with educational content, allowing users to progress through different levels by solving questions and challenges. The primary aim of the system is to combine entertainment with learning, ensuring that users remain interested while also gaining knowledge in a structured manner.

The system is developed using modern web technologies such as HTML, CSS, and JavaScript, which provide a flexible and dynamic environment for building interactive user interfaces. The frontend is designed with a horror aesthetic, incorporating dark themes, animations, and visual effects to create a unique atmosphere. JavaScript is used extensively to manage game logic, handle user interactions, and dynamically update content based on user actions. This ensures a smooth and responsive experience across different devices and browsers.

A key feature of the proposed system is its level-based progression model. Users begin at an initial level and advance by correctly answering multiple-choice questions and solving puzzles. Each level introduces increasing complexity, making the system both challenging and rewarding. The inclusion of puzzle elements, such as image-based challenges, adds variety to the gameplay and encourages critical thinking beyond simple question answering.

Another significant aspect of the system is the integration of audio elements, including sound effects and horror-themed voice narration. Sound effects are used during transitions, actions, and level changes to enhance immersion. Additionally, when a user answers incorrectly and loses all their lives, a voice narration provides the correct explanation in a dramatic and engaging manner.

The proposed system also includes a well-structured feedback mechanism. Users receive immediate responses to their actions, such as correct or incorrect answers, along with visual and audio cues. This real-time feedback helps users quickly identify their mistakes and learn from them. The system ensures that learning is continuous and interactive rather than passive, which significantly improves knowledge retention.

In terms of usability, the system is designed with a user-friendly interface and intuitive navigation controls. Features such as next, back, and menu options allow users to move smoothly between different sections of the application. The layout is responsive, ensuring compatibility with various screen sizes and devices. Special attention is given to maintaining a balance between aesthetic design and functionality, so that the interface remains both visually appealing and easy to use. Additionally, the proposed system is designed with scalability and future expansion in mind. The architecture allows easy integration of new features such as additional levels, advanced puzzles, and database connectivity for storing user progress and scores. It can also be extended to include user authentication, leaderboards, and performance analytics to enhance competitiveness and personalization. This flexibility ensures that the system can evolve over time, adapting to new requirements and technological advancements while continuing to provide an engaging and effective learning experience.

The architecture of the system is modular, consisting of different components such as the user interface module, quiz module, puzzle module, game logic module, navigation module, and audio feedback module. Each module performs a specific function, making the system easier to develop, test, and maintain. This modular approach also allows for future scalability, where new features and levels can be added without affecting the existing structure.

Overall, the proposed system offers a comprehensive solution to the limitations of traditional quiz platforms by introducing interactivity, immersion, and engagement. By combining gamification, storytelling, and multimedia elements, Nightmare Academy provides a unique and effective learning environment. The system not only enhances user experience but also ensures that learning becomes enjoyable, interactive, and impactful.

5.2 PROPOSED SYSTEM ARCHITECTURE

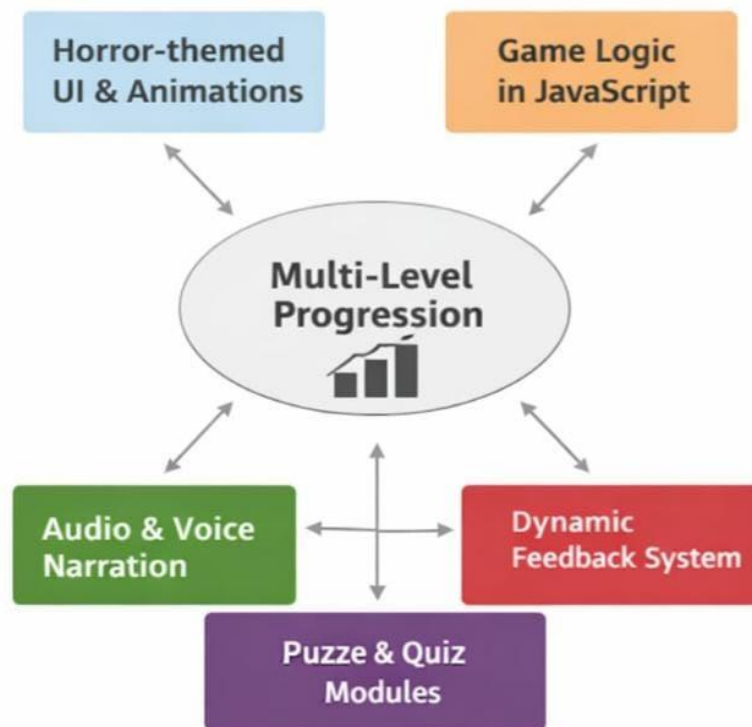


Figure 5.5 Proposed System

The proposed system architecture of Nightmare Academy follows a client-side web application model, where the entire functionality is primarily handled in the frontend using HTML, CSS, and JavaScript. The HTML structure defines the layout of the application, while CSS is used to create the horror-themed design with animations and visual effects. JavaScript acts as the core component, managing user interactions, game logic, and dynamic content updates. This architecture ensures a lightweight and responsive system that can run efficiently on standard web browsers without requiring heavy backend processing.

A special feature of the proposed system is the integration of puzzle-solving tasks before answering quiz questions. These small challenges help improve concentration, reasoning ability, and problem-solving skills. The system also provides different stages of difficulty such as beginner, intermediate, and advanced levels, allowing students to move forward based on their understanding and performance.

This step-by-step progression helps learners build confidence and stay motivated throughout the game. The horror-based design gives the system a unique identity by using dark themes, animated ghost characters, background videos, spooky sound effects, and suspenseful music. These features create an immersive atmosphere that keeps students engaged for a longer time. Developed using HTML, CSS, JavaScript, and backend database support, the system is easy to access through computers and laptops. The main purpose is to increase student involvement in studies by introducing fun elements such as challenges, rewards, levels, and visual storytelling, making the learning process less stressful and more exciting. Users can choose their preferred subject and begin solving quizzes in a game-like structure.

Overall, the proposed system offers a smart combination of education, entertainment, and technology to create a better and more memorable learning experience. The system is designed using a modular approach, where different functionalities are divided into separate modules such as the user interface module, quiz module, puzzle module, game logic module, navigation module, and audio feedback module. Each module operates independently but interacts with others to ensure smooth execution of the application.

5.3 ADVANTAGES

- Provides accurate 3D reconstruction of crime scenes using image data.
- Provides an interactive and engaging learning experience
- Uses gamification to increase user motivation and interest
- Incorporates horror theme for immersive storytelling
- Includes sound effects and voice narration for better engagement
- Offers instant feedback and explanations for wrong answers
- Supports level-based progression to maintain challenge and excitement
- Enhances knowledge retention through interactive learning
- Attractive and modern user interface design
- Easy to use with smooth navigation controls
- Scalable system that can be extended with more features in future

CHAPTER 6

SYSTEM REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

- **Processor**

A computer system with a modern processor such as Intel Core i5 or higher is required to run the proposed system efficiently. The processor handles image processing tasks, execution of backend scripts, and 3D reconstruction algorithms. A multi-core processor ensures smooth performance during feature extraction, reconstruction, and visualization processes.

- **Memory (RAM)**

The system requires a minimum of 8 GB RAM to execute the application without performance issues. Adequate memory is necessary for processing multiple images, running computer vision algorithms, and handling 3D model generation. Sufficient RAM ensures smooth multitasking and faster execution of reconstruction processes.

- **Storage**

A minimum of 256 GB SSD storage is required to store project files, reconstructed 3D models, and other records. SSD storage improves system speed and ensures faster access to large image datasets and 3D model files.

- **Device**

A system with small resolution for the localhost to process the questions and the answers in real time. A device with good sound system to hear the correct answer in the voice manipulation of a ghost if the user gives the wrong answer.

- **Internet Connectivity**

A stable internet connection is required for data transfer between the frontend and backend systems. It also supports uploading images, accessing APIs, and enabling communication between different modules of the system.

6.2 SOFTWARE REQUIREMENTS

- **Operating System**

The proposed system can be developed and executed on Windows or Linux operating systems. These platforms provide the necessary environment to run Python applications, databases, and 3D visualization tools efficiently.

- **Web browser**

A modern web browser such as Google Chrome, Microsoft Edge, or Mozilla Firefox is required. These browsers support the latest web technologies used in the project. They ensure smooth performance and proper rendering of UI and audio features.

- **Frontend technologies**

The application is developed using HTML, CSS, and JavaScript. HTML is used for structuring the content, while CSS handles styling and animations. JavaScript manages the logic, interactivity, and dynamic behavior of the system.

- **Code editor**

Visual Studio Code (VS Code) is used for developing the project. It provides useful features like syntax highlighting, extensions, and debugging tools. This helps in efficient coding and project management.

- **Local Server**

SQLite or A local server such as XAMPP is used to run the application. It helps in testing the project in a browser environment. It also ensures proper loading of files and smooth execution.

- **Audio Support**

The system uses the Web Audio API for integrating sound effects and voice narration. It allows playback of audio based on user actions. This enhances the immersive experience of the application.

- **Development Environment**

Visual Studio Code (VS Code) is used as the development environment for implementing the project. It provides tools for coding, debugging, and managing project files efficiently.

CHAPTER 7

SYSTEM IMPLEMENTATIONS

7.1 LIST OF MODULES

The implementation follows a modular approach, where each component of the system is developed separately and then integrated. Audio features such as sound effects and voice narration are implemented using web audio technologies, triggered based on user actions. The system ensures smooth navigation between levels, real-time feedback, and proper handling of user inputs, making the application interactive, responsive, and user-friendly.

The system is divided into multiple modules to ensure better organization and efficient functioning. Each module is responsible for a specific task, such as handling the user interface, managing quiz questions, controlling game logic, and processing user inputs. This separation of functionalities helps in simplifying the development process and allows each part of the system to be developed and tested independently.

The modular design also improves scalability and maintainability of the application. New features, such as additional levels, advanced puzzles, or enhanced audio effects, can be easily added without affecting the existing modules. This approach makes the system flexible, easy to upgrade, and suitable for future enhancements while maintaining smooth overall performance.

- User Interface Module
- Quiz Module
- Puzzle Module
- Game Logic Module
- Navigation Module
- Audio & Voice Feedback Module

7.2 MODULE DESCRIPTION

The “Nightmare Academy – Horror Quiz Game” consists of several important modules that work together to create an interactive learning platform. The first module is the User Module, where students can register, log in, and access the system securely. This module manages user details such as username, password, progress, and scores. It allows each student to have a personalized learning experience and helps track their performance throughout the game.

The second module is the Quiz and Subject Module, which contains different academic subjects and questions, checking answers, and updating scores according to user performance. The third module is the Puzzle and Challenge Module, which adds an extra layer of interaction before the quiz begins. Users must solve small puzzles, riddles, or challenges that improve concentration and logical thinking. The fourth module is the Horror Theme and Feedback Module, which creates the unique gaming atmosphere of the project.

7.2.1 User Interface Module

The User Interface Module is responsible for the overall look and feel of the application. It is designed using HTML and CSS to create a horror-themed environment with dark visuals, animations, and interactive elements. This module ensures that the interface is visually appealing and attracts users.

It also focuses on user experience by providing a simple and easy-to-use layout. Buttons, menus, and navigation elements are placed clearly so users can interact without confusion. The responsive design allows the system to work smoothly on different devices.

7.2.2 Quiz Module

The Quiz Module handles all the multiple-choice questions in the system. It displays questions along with options and allows users to select their answers. The module checks whether the selected answer is correct or incorrect.

It also provides immediate feedback to the user based on their response. This helps users understand their mistakes and improves their learning. The module is designed to make the quiz process smooth and interactive.

7.2.3 Puzzle Module

The Puzzle Module introduces additional challenges in the form of image-based or logical puzzles. Users are required to solve these puzzles to progress to the next level. This adds variety and makes the system more interesting.

The module enhances problem-solving skills by engaging users in interactive tasks. It breaks the monotony of regular quiz questions and keeps users motivated throughout the game.

7.2.4 Game Logic Module

The Game Logic Module controls the core functionality of the system. It manages levels, tracks user progress, and maintains the number of lives and scores. It also determines whether the user wins or loses.

This module ensures proper flow of the game by handling level transitions and conditions. It keeps the system organized and ensures that all actions are performed correctly based on user input.

7.2.5 Navigation Module

The Navigation Module manages the movement between different pages and levels in the application. It includes buttons like next, back, and menu to help users move easily through the system.

It ensures smooth transitions and prevents confusion during usage. This module improves usability by making the application simple and user-friendly.

7.2.6 Audio & Voice Feedback Module

The Audio & Voice Feedback Module handles all sound-related features in the system. It includes sound effects for actions, level changes, and background atmosphere to enhance the horror experience.

The voice explains the correct answer in a horror tone, making the learning process more engaging and memorable.

CHAPTER 8

SYSTEM TESTING

8.1 UNIT TESTING

Unit testing focuses on testing individual components or modules of the system separately. Each module, such as the quiz module or audio module, is tested independently to ensure it functions correctly without errors.

It also makes debugging easier since errors can be traced back to a specific module without affecting the entire system. For example, the login module is tested to ensure valid users can access the system and invalid entries are rejected properly. The quiz module is checked to verify that questions load correctly, answers are evaluated accurately, and scores are updated without mistakes.

8.2 INTEGRATION TESTING

Integration testing is performed after unit testing, where different modules are combined and tested together. It ensures that all modules, such as the quiz, game logic, and audio modules, work properly when integrated.

This testing helps in identifying issues related to data flow and communication between modules. In this phase, multiple modules are combined and tested together to ensure that they work properly as a unified system. For instance, after a user logs in, the system should successfully direct them to the subject selection page and then to the quiz section.

8.3 SYSTEM TESTING

System testing is carried out on the complete and fully integrated application. It checks whether the entire system meets the specified requirements and functions as expected in a real environment.

This testing ensures that all features, including UI, quiz functionality, puzzles, and audio effects, work together without any issues. The main purpose is to confirm that all functions work correctly without system crashes, missing features, or unexpected errors.

8.4 PERFORMANCE TESTING

Performance testing evaluates how the system behaves under different conditions, such as load and speed. It checks whether the application responds quickly and runs smoothly without delays. It also checks how the system behaves when multiple users access it at the same time. Performance testing helps developers identify slow sections, unnecessary delays, and memory issues. Improving these areas ensures the system remains fast, efficient, and user-friendly during continuous use.

This testing ensures that the system can handle multiple interactions efficiently. It also helps in identifying performance bottlenecks and improving system responsiveness. It is conducted to evaluate how well the system performs under different workloads. This includes measuring system response time, processing speed, and overall efficiency.

8.5 USABILITY TESTING

Usability testing focuses on how easy and user-friendly the application is. It checks whether users can navigate the system easily and understand how to use its features without confusion. It also helps in examining button placement, menu structure, instructions, font readability, and overall screen design.

For example, if users struggle to find the next level button or misunderstand puzzle instructions, changes can be made to improve the interface. Usability testing ensures that the horror theme remains attractive without affecting readability or user comfort. This testing helps improve the overall user experience by identifying design issues and making necessary improvements. It ensures that the interface is simple, clear, and accessible for all users.

CHAPTER 9

RESULT AND DISCUSSION

The developed system, Nightmare Academy, was successfully implemented as a web-based interactive quiz application. The puzzle module added variety to the application by introducing non-traditional challenges. Users were able to interact with image-based puzzles, which enhanced their problem-solving skills. This feature helped break the monotony of regular quiz questions and kept the experience dynamic.

The game logic module effectively managed user progress, including tracking lives, handling win/lose conditions, and controlling level transitions. It ensured that the flow of the application remained consistent and error-free. This contributed to a smooth gaming experience. Navigation within the system was simple and intuitive, allowing users to move between different sections without confusion. Buttons and controls were responsive and clearly designed. This improved usability and reduced the learning curve for new users.

The audio and voice feedback module significantly enhanced the immersive experience of the system. Sound effects during actions and transitions created a horror atmosphere, while the voice explanations provided clarity in an engaging manner. This combination made the system more interactive compared to traditional methods.

Finally, the overall system proved to be reliable and efficient during testing. It successfully combined education with entertainment, achieving the main objective of the project. It was tested on multiple platforms, and the performance remained consistent without major layout or functionality issues. This ensures that users can access the application easily without requiring specific configurations.

Overall, the results indicate that the proposed system successfully overcomes the limitations of traditional quiz applications. It provides a more engaging, interactive, and immersive learning experience. The combination of gamification, storytelling, and multimedia elements makes the system effective in improving user interest and knowledge retention.

During testing, the application demonstrated smooth performance and responsiveness. The user interface was visually appealing and aligned with the horror theme, which helped in capturing user attention. Animations and transitions worked properly, enhancing the overall experience without affecting performance. One of the key highlights of the system is the integration of audio features.

Sound effects and horror voice narration worked efficiently, especially when users answered incorrectly and lost their lives. The voice-based explanation provided an engaging way to deliver correct answers, making the learning process more interactive and memorable.

The modular architecture of the system proved to be effective in ensuring proper functionality and easy maintenance. Each module, such as quiz, puzzle, game logic, and audio, worked independently and in coordination with others. The system also showed good compatibility across different web browsers and devices. It was tested on multiple platforms, and the performance remained consistent without major layout or functionality issues. This ensures that users can access the application easily without requiring specific configurations.

Error handling mechanisms in the system worked effectively during testing. Invalid inputs and unexpected user actions were managed properly without causing crashes. This improved the stability of the application and ensured a smooth user experience even in edge cases.

User feedback indicated that the application was both enjoyable and informative. The results show that the system can be further expanded and improved, making it a strong foundation for future developments. The combination of horror elements with educational content made the experience unique and engaging. Many users found the voice explanations and interactive design helpful in understanding concepts better and retaining information.

CHAPTER 10

CONCLUSION AND FUTURE ENHANCEMENTS

10.1 CONCLUSION

The project Nightmare Academy was successfully designed and implemented as an interactive web-based quiz application. It effectively combines education with entertainment through a horror-themed environment. The system achieved its primary goal of making learning more engaging and enjoyable for users. The application integrates multiple features such as MCQs, puzzles, level-based progression, and audio feedback. These features work together to create a dynamic and immersive user experience. The inclusion of sound effects and voice narration further enhances user interaction and understanding.

The project also highlights the importance of combining creativity with technical knowledge. By integrating a horror theme with educational content, the system demonstrates how innovative ideas can improve traditional learning methods. This approach makes the application unique and more appealing to users. In addition, the successful implementation of web technologies and multimedia elements proves that interactive systems can be developed efficiently without complex tools. The project serves as a strong example of how simple technologies can be used to build effective and engaging applications.

The modular design of the system made development and testing easier. Each module functioned independently while maintaining smooth coordination with other components. This approach ensured stability, flexibility, and ease of maintenance. The system also demonstrated good performance and usability during testing. It provided quick responses, smooth navigation, and an attractive interface. Users were able to interact with the system easily without facing major difficulties. Overall, the project successfully addresses the limitations of traditional quiz systems. It introduces a new way of learning by combining gamification, storytelling, and multimedia elements.

10.2 FUTURE ENHANCEMENTS

In the future, the system can be enhanced by integrating a database to store user data and progress. This would allow users to log in, track their scores, and continue from where they left off. It would also improve personalization of the learning experience. Another enhancement would be the addition of multiplayer functionality. Users could compete with friends or other players in real time, making the system more interactive and competitive. This would increase user engagement and motivation. The system can also be improved by adding more levels and advanced puzzles. Introducing different categories of questions and difficulty levels would make the application more diverse and challenging.

This would help in expanding its usability for different types of users. Advanced audio features, such as AI-based voice interaction, can be implemented to make the system more interactive. Users could receive real-time responses or even communicate with the system. This would further enhance the immersive experience. The system can be enhanced by incorporating more advanced visual effects and 3D elements to improve the horror experience. Adding features like dynamic lighting, animations, and interactive environments can make the application more immersive. This would further engage users and create a more realistic gameplay atmosphere.

Another improvement would be the integration of voice recognition technology. Users could interact with the system using voice commands, making the experience more natural and interactive. This would also improve accessibility for users who prefer hands-free interaction. The application can also be expanded to support multiple languages. This would make the system accessible to a wider audience and increase its usability across different regions. Providing localized content and voice narration in different languages would enhance user understanding and engagement. Finally, the application can be extended to mobile platforms or converted into a full-fledged mobile app.

APPENDIX – A

SOURCE CODE

Style.css

```
:root {
    --bg-dark: #2a2d34;
    --blood-red: #8a0303;
    --sickly-green: #2e4a2e;
    --ghost-white: rgba(255, 255, 255, 0.7);
    --text-glow: 0 0 10px #ff0000, 0 0 20px #ff0000;
}

body, html {
    margin: 0;
    padding: 0;
    width: 100%;
    height: 100%;
    background-color: var(--bg-dark);
    color: #ddd;
    font-family: 'Courier New', Courier, monospace;
    overflow: hidden;
}

/* Background Noise */
#noise-overlay {
    position: fixed;
    top: 0; left: 0; right: 0; bottom: 0;

    background-image: url('data:image/svg+xml,%3Csvg viewBox="0 0 200 200"
xmlns="http://www.w3.org/2000/svg"%3E%3Cfilter id="noiseFilter"%3E%3CfeTurbulence
type="fractalNoise" baseFrequency="0.65" numOctaves="3"
stitchTiles="stitch"/%3E%3C/filter%3E%3Crect width="100%25" height="100%25"
filter="url(%23noiseFilter)"/%3E%3C/svg%3E');

    opacity: 0.05;
    z-index: 999;
    pointer-events: none;
}
```

```

}
#bg-video {
    position: fixed;
    top: 50%;
    left: 50%;
    min-width: 100%;
    min-height: 100%;
    width: auto;
    height: auto;
    transform: translateX(-50%) translateY(-50%);
    z-index: 1;
    object-fit: cover;
    opacity: 0.6;
}
#vignette {
    position: fixed;
    top: 0; left: 0; right: 0; bottom: 0;
    background: radial-gradient(circle, transparent 30%, rgba(5,0,0,0.95) 100%);
    box-shadow: inset 0 0 100px rgba(0,0,0,0.9);
    z-index: 998;
    pointer-events: none;
}
/* Enhanced Background Effects */
#fog-layer {
    position: fixed;
    top: 0; left: -100vw; width: 300vw; height: 100vh;
    background: transparent
url('https://upload.wikimedia.org/wikipedia/commons/e/e0/Clouds.jpg') repeat-x;
    background-size: cover;
    opacity: 0.25;
    mix-blend-mode: color-burn;
    filter: sepia(100%) hue-rotate(100deg) blur(5px);
    z-index: 0;

```

```

    pointer-events: none;
    animation: fogDrift 60s linear infinite;
}
@keyframes fogDrift {
    0% { transform: translateX(0); }
    100% { transform: translateX(-100vw); }
}
#souls-container {
    position: fixed;
    top: 0; left: 0; right: 0; bottom: 0;
    z-index: 1;
    overflow: hidden;
    pointer-events: none;
}
.soul {
    position: absolute;
    bottom: -50px;
    width: 5px;
    height: 15px;
    border-radius: 50%;
    background: rgba(100, 255, 100, 0.6);
    box-shadow: 0 0 15px 5px rgba(50, 200, 50, 0.8);
    filter: blur(2px);
    opacity: 0;
    animation: soulFloat 5s ease-in infinite;
}
@keyframes soulFloat {
    0% { transform: translateY(0) scale(1); opacity: 0; }
    20% { opacity: 0.8; }
    80% { opacity: 0.6; }
    100% { transform: translateY(-100vh) scale(0.5); opacity: 0; }
}

```

```
#app-container {
  position: relative;
  width: 100%;
  height: 100%;
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 10;
}

.screen {
  width: 100%;
  height: 100%;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  position: absolute;
  top: 0;
  left: 0;
  opacity: 0;
  pointer-events: none;
  transition: opacity 1s ease-in-out;
}

.screen.active {
  opacity: 1;
  pointer-events: all;
}

.hidden {
  display: none !important;
}

/* Fonts */

.glitch {
```

```
font-family: 'Nosifer', cursive;
color: var(--blood-red);
font-size: 4rem;
position: relative;
text-shadow: 2px 2px 5px black;
animation: pulseGlow 2s infinite;
text-align: center;
line-height: 1.2;
}

.subtitle {
  font-family: 'Creepster', cursive;
  font-size: 2rem;
  color: #555;
  letter-spacing: 5px;
  margin-bottom: 2rem;
}

.blood-text {
  font-family: 'Creepster', cursive;
  color: var(--blood-red);
  font-size: 3rem;
  text-shadow: var(--text-glow);
}

.spooky-text {
  font-family: 'Jolly Lodger', cursive;
  color: var(--ghost-white);
  font-size: 2.5rem;
  text-align: center;
  max-width: 800px;
  padding: 20px;
  background: rgba(0,0,0,0.6);
  border-radius: 10px;
```

```

    border: 1px solid #333;
}
/* Buttons */
.blood-btn {
    background: rgba(138, 3, 3, 0.2);
    border: 2px solid #ff3333;
    color: #ffdddd;
    font-family: 'Courier New', Courier, monospace;
    font-size: 1.5rem;
    font-weight: bold;
    padding: 15px 30px;
    margin: 10px;
    cursor: pointer;
    position: relative;
    overflow: hidden;
    transition: all 0.3s;
    text-transform: uppercase;
    text-shadow: 1px 1px 3px #000;
}
.blood-btn:hover {
    background: var(--blood-red);
    color: #fff;
    box-shadow: 0 0 15px #ff3333;
}
/* Ghost Images */
.ghost-img {
    width: 200px;
    height: 200px;
    border-radius: 50%;
    object-fit: cover;
    filter: grayscale(50%) contrast(120%) brightness(90%) drop-shadow(0 0 20px var(--sickly-green));
}

```



```

    animation: float 4s ease-in-out infinite;
    mix-blend-mode: screen;
}

.ghost-img.angry {
    width: 120px;
    height: 120px;
    filter: invert(100%) hue-rotate(180deg) drop-shadow(0 0 40px red);
    animation: shake 0.5s infinite;
}

.ghost-bg {
    width: 600px;
    height: 600px;
    object-fit: cover;
    opacity: 0.4;
    filter: grayscale(50%) blur(1px);
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    pointer-events: none;
    z-index: -1;
    animation: breathes 6s infinite alternate;
}

/* Animations */

@keyframes float {
    0% { transform: translateY(0px); opacity: 0.8; }
    50% { transform: translateY(-20px); opacity: 1; }
    100% { transform: translateY(0px); opacity: 0.8; }
}

@keyframes breathes {
    0% { transform: translate(-50%, -50%) scale(1); opacity: 0.1; }
    100% { transform: translate(-50%, -50%) scale(1.1); opacity: 0.25; }
}

```

```

}
@keyframes pulseGlow {
  0% { text-shadow: 0 0 5px red; }
  50% { text-shadow: 0 0 20px red, 0 0 30px darkred; }
  100% { text-shadow: 0 0 5px red; }
}
@keyframes shake {
  0% { transform: translate(1px, 1px) rotate(0deg); }
  10% { transform: translate(-1px, -2px) rotate(-1deg); }
  20% { transform: translate(-3px, 0px) rotate(1deg); }
  30% { transform: translate(3px, 2px) rotate(0deg); }
  40% { transform: translate(1px, -1px) rotate(1deg); }
  50% { transform: translate(-1px, 2px) rotate(-1deg); }
  60% { transform: translate(-3px, 1px) rotate(0deg); }
  70% { transform: translate(3px, 1px) rotate(-1deg); }
  80% { transform: translate(-1px, -1px) rotate(1deg); }
  90% { transform: translate(1px, 2px) rotate(0deg); }
  100% { transform: translate(1px, -2px) rotate(-1deg); }
}
/* Quiz UI */
.hud {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  padding: 10px 30px;
  box-sizing: border-box;
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: rgba(10, 0, 0, 0.95);
  border-bottom: 2px solid var(--blood-red);

```

```

    box-shadow: 0 5px 15px rgba(138, 3, 3, 0.4);
    font-family: 'Creepster', cursive;
    font-size: 2.2rem;
    color: #fff;
    text-shadow: 2px 2px 5px #000;
    z-index: 1000;
}
#lives-display {
    color: #ff3333;
    text-shadow: 0 0 10px #ff0000, 2px 2px 5px #000;
}
/* Puzzle UI */
.puzzle-grid {
    display: grid;
    grid-template-columns: repeat(3, 100px);
    grid-template-rows: repeat(3, 100px);
    gap: 2px;
    background: #111;
    padding: 5px;
    border: 2px solid var(--blood-red);
    box-shadow: 0 0 20px var(--blood-red);
    margin: 0;
}
.puzzle-tile {
    width: 100px;
    height: 100px;
    background-size: 300px 300px;
    cursor: pointer;
    transition: transform 0.2s;
}
.puzzle-tile:hover {
    transform: scale(1.05);
}

```

```
    z-index: 10;
    box-shadow: 0 0 10px white;
}

.quiz-container {
    width: 80%;
    max-width: 800px;
    background: rgba(10, 0, 0, 0.8);
    border: 1px solid var(--blood-red);
    border-radius: 5px;
    padding: 40px;
    text-align: center;
    box-shadow: 0 0 30px rgba(138, 3, 3, 0.4);
}

.question-box {
    background: rgba(0,0,0,0.85); /* Readability over video */
    font-size: 1.5rem;
    margin-bottom: 30px;
    line-height: 1.4;
    padding: 10px;
    border-radius: 5px;
}

.options-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 15px;
}

.option-btn {
    background: rgba(0,0,0,0.85); /* High contrast */
    border: 1px solid #444;
    color: #ccc;
    padding: 15px;
    font-size: 1.2rem;
```

```

    cursor: pointer;
    transition: 0.2s;
}
.option-btn:hover {
    background: #222;
    border-color: var(--sickly-green);
    color: #fff;
    box-shadow: 0 0 10px var(--sickly-green);
}
/* Jump Scare */
#scare-screen {
    background: black;
    z-index: 1000;
}
.scare-face {
    width: 100vw;
    height: 100vh;
    background-image:
url('https://upload.wikimedia.org/wikipedia/commons/thumb/4/43/The_Ghost.jpg/800px-The_Ghost.jpg');
    background-size: cover;
    background-position: center;
    filter: contrast(200%) brightness(150%) hue-rotate(10deg);
    animation: flash 0.1s infinite;
}
@keyframes flash {
    0% { filter: invert(0%); }
    50% { filter: invert(100%); }
}
/* Lesson Box Overlay */
.lesson-box {
    background: rgba(0, 0, 0, 0.85);

```

```
padding: 15px;
border: 1px solid var(--blood-red);
border-radius: 5px;
margin: 10px auto;
max-width: 600px;
}
```

App.js

```
// Game State
```

```
let currentState = {
  domain: null, // 'web', 'db', 'net', 'os', 'ai'
  difficulty: null, // 'easy', 'medium', 'hard'
  maxLives: 3,
  level: 1, // 1 to 5
  questionIndex: 0, // 0 to 2 for current level
  lives: 3
};
```

```
// Ghost appearances per level (we use CSS filter adjustments on a base image)
```

```
const ghostHues = [0, 90, 150, 220, 280];
```

```
// Questions are now loaded from js/questions.js and js/data/*
```

```
function getCurrentQuestionData() {
  const totalQIdx = (currentState.level - 1) * 3 + currentState.questionIndex;
  return gameTriviaQuestions[currentState.difficulty][currentState.domain][totalQIdx];
}
```

```
// UI Handling
```

```
function showScreen(id) {
  if (window.speechSynthesis) window.speechSynthesis.cancel();
  document.querySelectorAll('.screen').forEach(s => s.classList.remove('active'));
  setTimeout(() => {
    document.querySelectorAll('.screen').forEach(s => s.classList.add('hidden'));
    const screen = document.getElementById(id);
    screen.classList.remove('hidden');
    setTimeout(() => screen.classList.add('active'), 50);
  });
}
```

```

    }, 500); // fade time
}
function triggerJumpScare() {
    const scareStr = document.getElementById("scare-screen");
    scareStr.classList.remove("hidden");
    setTimeout(() => scareStr.classList.add("active"), 10);
    setTimeout(() => {
        scareStr.classList.remove("active");
        setTimeout(() => scareStr.classList.add("hidden"), 200);
    }, 400); // scare duration
}
function speakHorror(text) {
    if ('speechSynthesis' in window) {
        window.speechSynthesis.cancel();
        let spell = new SpeechSynthesisUtterance(text);
        spell.pitch = 0.1;
        spell.rate = 0.8;
        spell.volume = 1;
        // Duck the BGM music so the voice is loud
        const bgm = document.getElementById('horror-bgm');
        if (bgm) bgm.volume = 0.15;
        spell.onend = function() {
            if (bgm) bgm.volume = 0.5;
        };
        const voices = window.speechSynthesis.getVoices();
        const deepVoice = voices.find(v => v.name.toLowerCase().includes('david') ||
v.name.toLowerCase().includes('male'));
        if (deepVoice) spell.voice = deepVoice;
        window.speechSynthesis.speak(spell);
    }
}
// Pre-load voices to avoid silent first-try

```

```

if ('speechSynthesis' in window) {
  window.speechSynthesis.onvoiceschanged = () => window.speechSynthesis.getVoices();
}

function typeWriterEffect(element, text, speed = 50, callback = null) {
  element.innerHTML = "";
  let i = 0;
  if (element.typewriterInterval) clearInterval(element.typewriterInterval);
  element.typewriterInterval = setInterval(() => {
    if (i < text.length) {
      element.innerHTML += text.charAt(i);
      i++;
    } else {
      clearInterval(element.typewriterInterval);
      if (callback) callback();
    }
  }, speed);
}

function updateGhostColors() {
  const hue = ghostHues[currentState.level - 1];

  document.getElementById('intro-ghost-img').style.filter = `hue-rotate(${hue}deg)
  grayscale(50%) contrast(120%) brightness(100%) drop-shadow(0 0 20px var(--sickly-
  green))`;

  document.getElementById('quiz-ghost-img').style.filter = `hue-rotate(${hue}deg)
  grayscale(50%) opacity(0.5) blur(1px)`;

  // Removed invert(100%) to keep the 'Failed' ghost features visible

  document.getElementById('lesson-ghost-img').style.filter = `grayscale(100%) sepia(100%)
  hue-rotate(300deg) contrast(150%) brightness(80%) drop-shadow(0 0 40px red)`;

  // Change environment tint

  document.getElementById('fog-layer').style.filter = `sepia(100%) hue-rotate(${hue +
  40}deg) blur(5px)`;

  document.querySelectorAll('.soul').forEach(soul => {
    soul.style.filter = `hue-rotate(${hue}deg) blur(2px)`;
  });
}

```



```

}
// Emits ambient floating particles
function spawnSouls() {
  const container = document.getElementById('souls-container');
  setInterval(() => {
    const soul = document.createElement('div');
    soul.classList.add('soul');
    soul.style.left = Math.random() * 100 + 'vw';
    soul.style.animationDuration = (Math.random() * 4 + 3) + 's'; // 3 to 7 seconds
    // Match current level hue if game is active
    const currentHue = currentState.level ? ghostHues[currentState.level - 1] : 0;
    soul.style.filter = `hue-rotate(${currentHue}deg) blur(2px)`;
    container.appendChild(soul);
    // Remove after animation completes
    setTimeout(() => soul.remove(), 7000);
  }, 400); // spawn a new soul every 400ms
}
// Start ambient background
spawnSouls();
// Game Flow
function startGame(difficulty) {
  currentState.domain = document.getElementById('domain-select').value;
  currentState.difficulty = difficulty;
  // All difficulties have exactly 3 lives
  currentState.maxLives = 3;
  currentState.level = 1;
  currentState.questionIndex = 0;
  currentState.lives = currentState.maxLives;
  document.getElementById('home-btn').classList.remove('hidden');
  document.getElementById('global-hud').classList.remove('hidden');
  // Play the atmospheric horror music
  const bgm = document.getElementById('horror-bgm');

```

```

    if (bgm) {
        bgm.volume = 0.5;
        bgm.play().catch(e => console.log("BGM Autoplay prevented:", e));
    }
    triggerJumpScare();
    setTimeout(() => prepareLevel(), 500);
}

function returnToHome() {
    currentState.domain = null;
    document.getElementById('home-btn').classList.add('hidden');
    document.getElementById('global-hud').classList.add('hidden');
    // Reset fog and soul tints to default
    document.getElementById('fog-layer').style.filter = `sepia(100%) hue-rotate(100deg) blur(5px)`;
    // Stop the background music
    const bgm = document.getElementById('horror-bgm');
    if (bgm) {
        bgm.pause();
        bgm.currentTime = 0;
    }
    showScreen('login-screen');
}

function goBack() {
    if (document.getElementById('quiz-screen').classList.contains('active')) {
        showScreen('puzzle-screen');
    } else if (document.getElementById('puzzle-screen').classList.contains('active')) {
        showScreen('intro-screen');
    } else if (document.getElementById('intro-screen').classList.contains('active')) {
        returnToHome();
    }
}

function prepareLevel() {

```

```

if (currentState.level > 5) {
    document.getElementById('global-hud').classList.add('hidden');
    showScreen('win-screen');
    return;
}

currentState.questionIndex = 0;
currentState.lives = currentState.maxLives;

document.getElementById('intro-level-title').innerText = `Level ${currentState.level} :
${currentState.domain.toUpperCase()}`;

// Set matching ghost image
const ghostNum = currentState.level <= 5 ? currentState.level : 5; // Use 5th ghost for level
5

const ghostSrc = `images/ghost${ghostNum}.jpg.jpeg`;
document.getElementById('intro-ghost-img').src = ghostSrc;
document.getElementById('quiz-ghost-img').src = ghostSrc;
document.getElementById('lesson-ghost-img').src = ghostSrc;
const iElem = document.getElementById('intro-text');
iElem.innerHTML = ""; // Clear for animation
updateGhostColors();
showScreen('intro-screen');

setTimeout(() => typeWriterEffect(iElem, "I am the guardian of this knowledge...", 50),
500);
}

// --- PUZZLE LOGIC ---
let puzzleState = [];
let selectedTile = null;

function startPuzzle() {
    showScreen('puzzle-screen');

    // Map to puzzle1.jpg through puzzle15.jpg based on Level (1-5) and Question (0-2)
    const puzzleNum = (currentState.level - 1) * 3 + currentState.questionIndex + 1;
    const jpegPuzzles = [6, 7, 8, 9, 10, 11];
    const extension = jpegPuzzles.includes(puzzleNum) ? '.jpg.jpeg' : '.jpg';
    const imgSrc = `images/puzzle${puzzleNum}${extension}`;

```

```

document.getElementById('puzzle-ref-img').src = imgSrc;
const pElem = document.getElementById('puzzle-instruction-text');
pElem.innerHTML = "";
setTimeout(() => typeWriterEffect(pElem, "Swap the fragmented pieces to reveal the
anomaly before it asks its question...", 35), 500);
puzzleState = [0, 1, 2, 3, 4, 5, 6, 7, 8];
do {
  for (let i = puzzleState.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [puzzleState[i], puzzleState[j]] = [puzzleState[j], puzzleState[i]];
  }
} while (isPuzzleSolved()); // Ensure it's shuffled
selectedTile = null;
renderPuzzle(imgSrc);
}
function isPuzzleSolved() {
  for(let i=0; i<9; i++) {
    if (puzzleState[i] !== i) return false;
  }
  return true;
}
function renderPuzzle(imgSrc) {
  const grid = document.getElementById('puzzle-grid');
  grid.innerHTML = "";
  puzzleState.forEach((pos, index) => {
    const tile = document.createElement('div');
    tile.className = 'puzzle-tile';
    tile.style.backgroundImage = `url('${imgSrc}')`;
    const origX = (pos % 3) * 100;
    const origY = Math.floor(pos / 3) * 100;
    tile.style.backgroundPosition = `-${origX}px -${origY}px`;
    tile.style.filter = `contrast(110%) brightness(90%)`;
  });
}

```

```

    if (selectedTile === index) {
        tile.style.border = "2px solid red";
        tile.style.transform = "scale(0.9)";
    }
    tile.onclick = () => handleTileClick(index, imgSrc);
    grid.appendChild(tile);
  });
}

function handleTileClick(index, imgSrc) {
  if (selectedTile === null) {
    selectedTile = index;
  } else {
    const temp = puzzleState[selectedTile];
    puzzleState[selectedTile] = puzzleState[index];
    puzzleState[index] = temp;
    selectedTile = null;
  }
  renderPuzzle(imgSrc);
  if (isPuzzleSolved()) {
    setTimeout(() => {
      startQuiz();
    }, 500);
  }
}

function startQuiz() {
  showScreen('quiz-screen');
  renderHUD();
  loadQuestion();
}

function renderHUD() {
  document.getElementById('hud-level').innerText = currentState.level;
  const lost = 3 - currentState.lives;

```

```

    document.getElementById('lives-display').innerText = `Remaining: ${currentState.lives} |
Lost: ${lost}`;
}
function loadQuestion() {
    const qData = getCurrentQuestionData();
    const qElem = document.getElementById('question-text');
    const optionsContainer = document.getElementById('options-container');
    optionsContainer.innerHTML = "";
    optionsContainer.classList.add('hidden'); // Hide options until typed
    typeWriterEffect(qElem, qData.q, 45, () => {
        qData.opts.forEach((opt, idx) => {
            const btn = document.createElement('button');
            btn.className = 'option-btn';
            btn.innerText = opt;
            btn.onclick = () => checkAnswer(idx);
            optionsContainer.appendChild(btn);
        });
        optionsContainer.classList.remove('hidden');
    });
}
function checkAnswer(selectedIdx) {
    const qData = getCurrentQuestionData();
    if (selectedIdx === qData.ans) {
        // Correct
        speakHorror("Correct.");
        document.body.style.backgroundColor = '#050';
        setTimeout(() => document.body.style.backgroundColor = 'var(--bg-dark)', 200);
        currentState.questionIndex++;
        if (currentState.questionIndex >= 3) {
            // Level complete
            triggerJumpScare();
            currentState.level++;
        }
    }
}

```

```

        setTimeout(() => prepareLevel(), 1000);
    } else {
        // Trigger the puzzle again before the next question!
        startPuzzle();
    }
} else {
    // Wrong
    speakHorror("Wrong.");
    loseLife();
}
}
function loseLife() {
    currentState.lives--;
    renderHUD();
    // Flashing effect
    document.body.style.backgroundColor = '#500';
    setTimeout(() => document.body.style.backgroundColor = 'var(--bg-dark)', 200);
    if (currentState.lives <= 0) {
        // Show lesson only when completely out of lives
        setTimeout(() => showLesson(), 500);
    } else {
        // Optional quick scare on failure if they still have lives
        if(Math.random() > 0.5) triggerJumpScare();
    }
}
function showLesson() {
    // Fetch the specific lesson for this exact question!
    const qData = getCurrentQuestionData();
    const lessonTxt = qData.lesson;
    const lElem = document.getElementById('lesson-text');
    lElem.innerHTML = "";
    const titleElem = document.getElementById('lesson-title');

```

```

titleElem.innerText = "YOU FAILED";
titleElem.dataset.text = "YOU FAILED";
showScreen('lesson-screen');
speakHorror(lessonTxt);
setTimeout(() => typeWriterEffect(lElem, lessonTxt, 40), 500);
}

function handleLessonConfirm() {
  // Restore exactly 1 life after reading the lesson
  currentState.lives = 1;
  renderHUD();
  // Send them back to the exact same question they failed on!
  showScreen('quiz-screen');
}

function restartLevel() {
  handleLessonConfirm();
}

// Start BGM on first interaction
document.addEventListener('click', function initBGM() {
  const bgm = document.getElementById('horror-bgm');
  if (bgm && bgm.paused) {
    bgm.volume = 0.5;
    bgm.play().catch(e => console.log("Init BGM prevented:", e));
  }
  document.removeEventListener('click', initBGM);
}, { once: true });

```

Question.js

```

const gameTriviaQuestions = {
  easy: {
    web: webQuestions.easy,
    db: dbQuestions.easy,
    net: netQuestions.easy,
    os: osQuestions.easy,

```



```
    ai: aiQuestions.easy
  },
  medium: {
    web: webQuestions.medium,
    db: dbQuestions.medium,
    net: netQuestions.medium,
    os: osQuestions.medium,
    ai: aiQuestions.medium
  },
  hard: {
    web: webQuestions.hard,
    db: dbQuestions.hard,
    net: netQuestions.hard,
    os: osQuestions.hard,
    ai: aiQuestions.hard
  }
};
```

APPENDIX – B

SCREENSHOTS

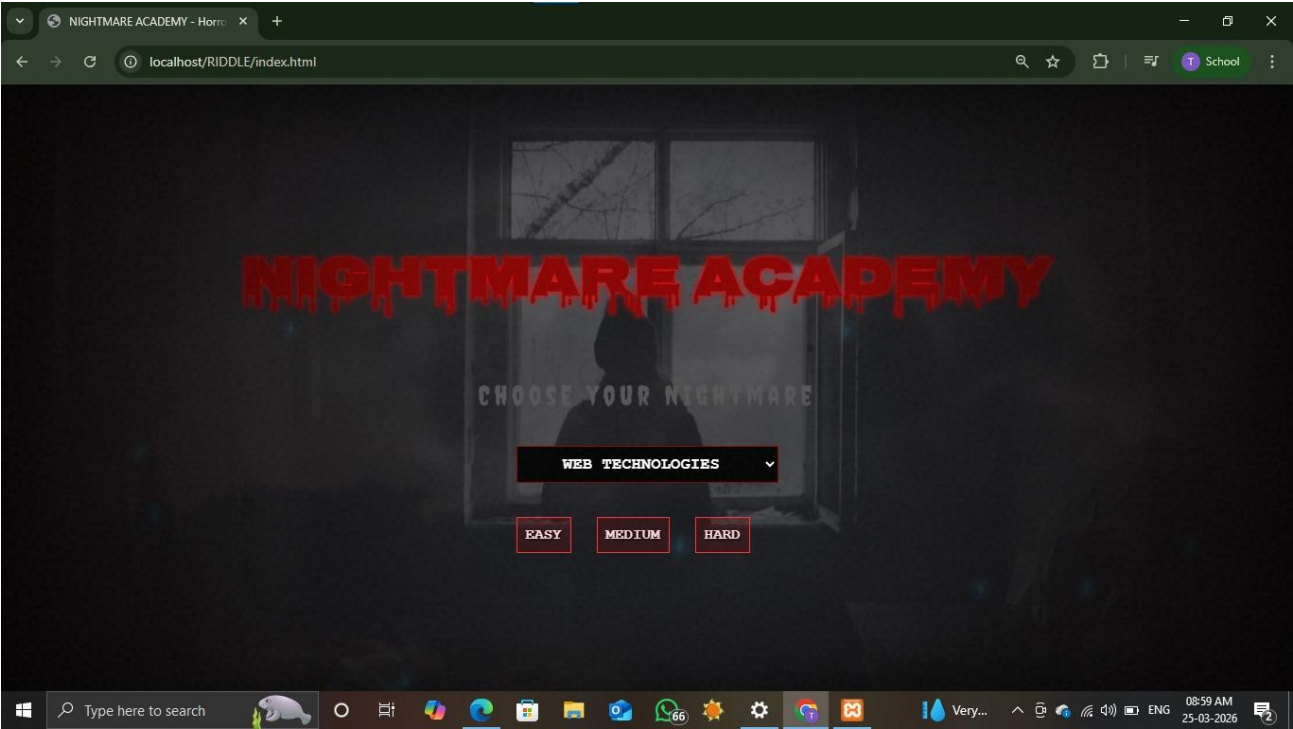


Figure B.1 Home Page

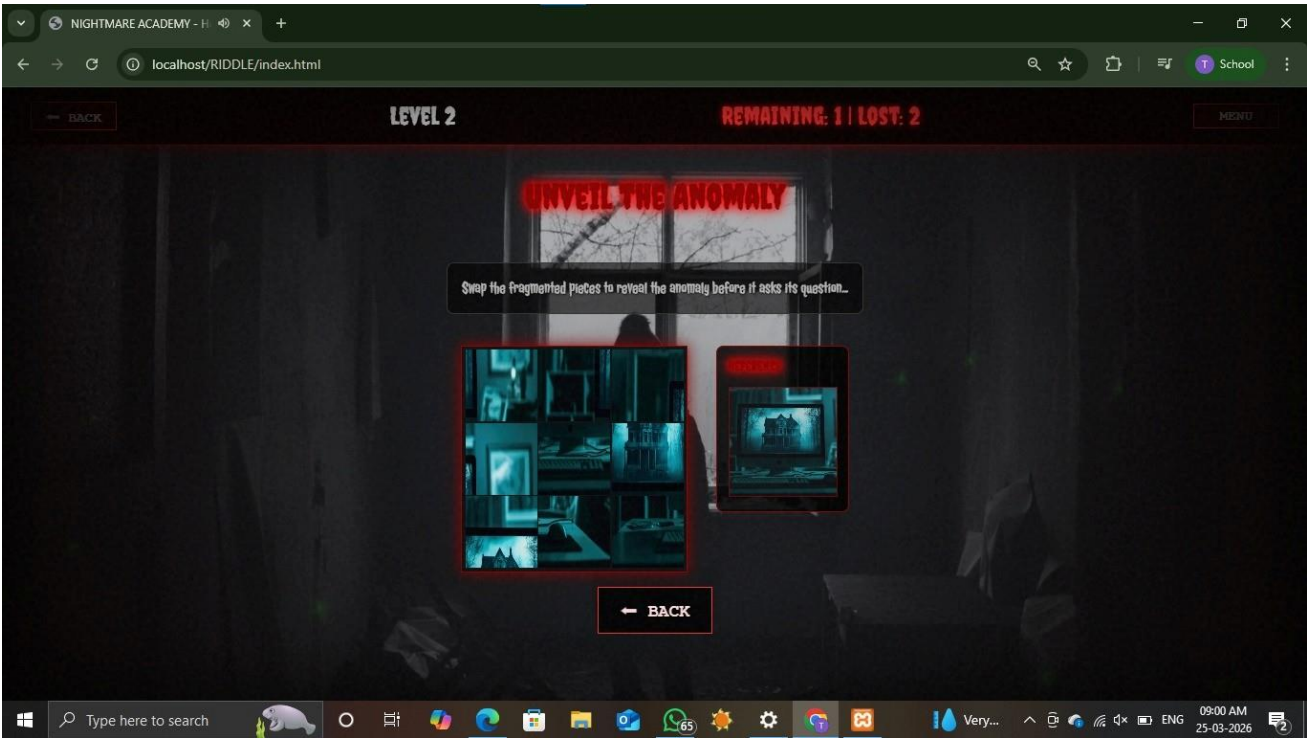


Figure B.2 Puzzle Page

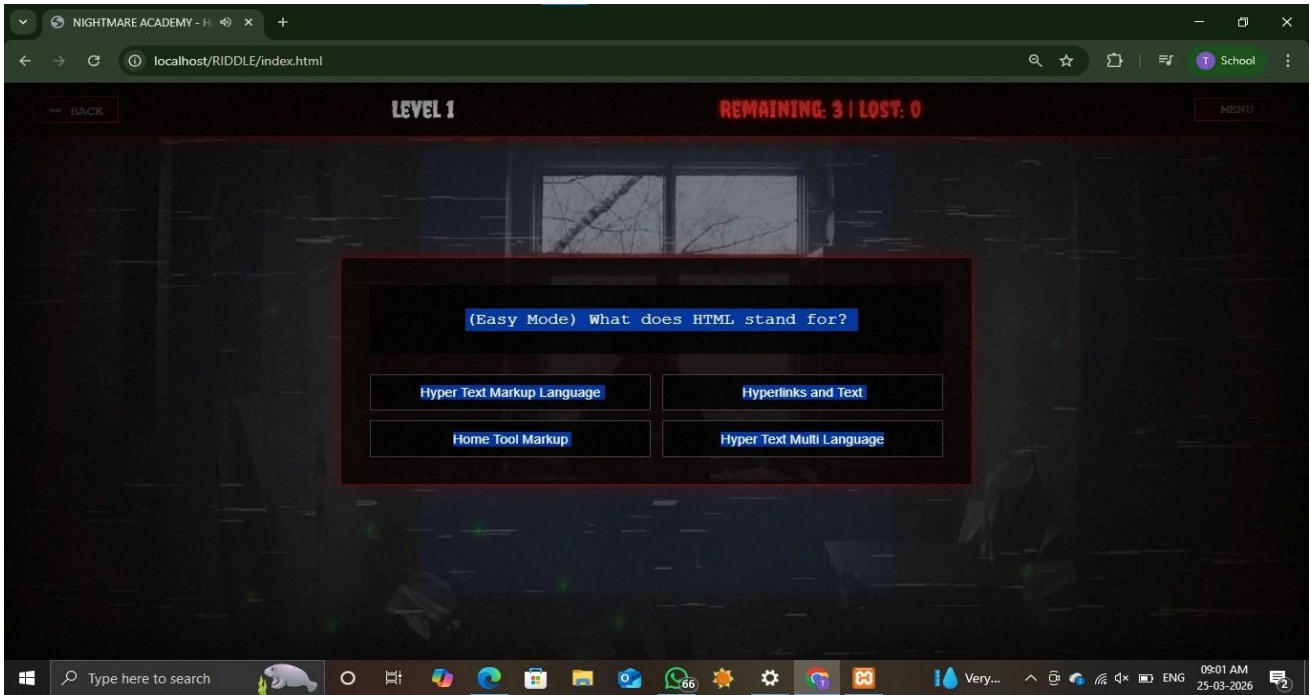


Figure B.3 Quiz Page

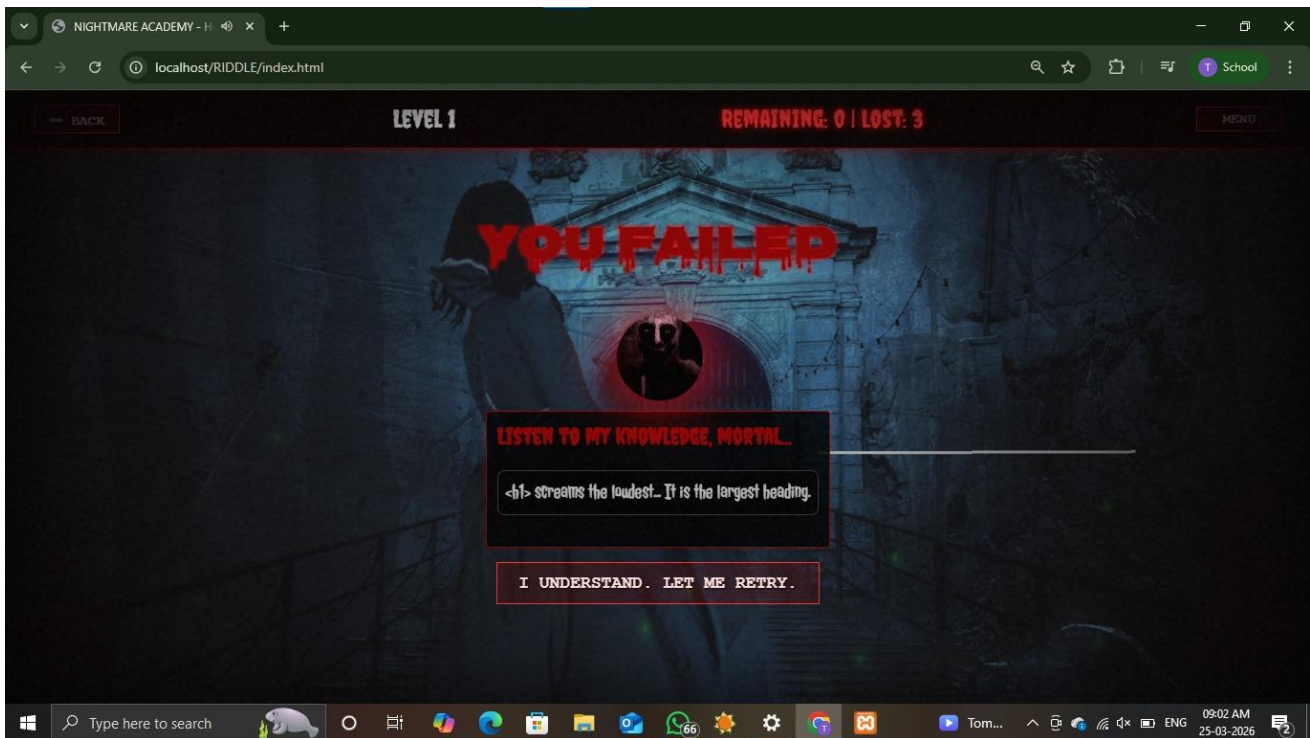


Figure B.4 game logic Page

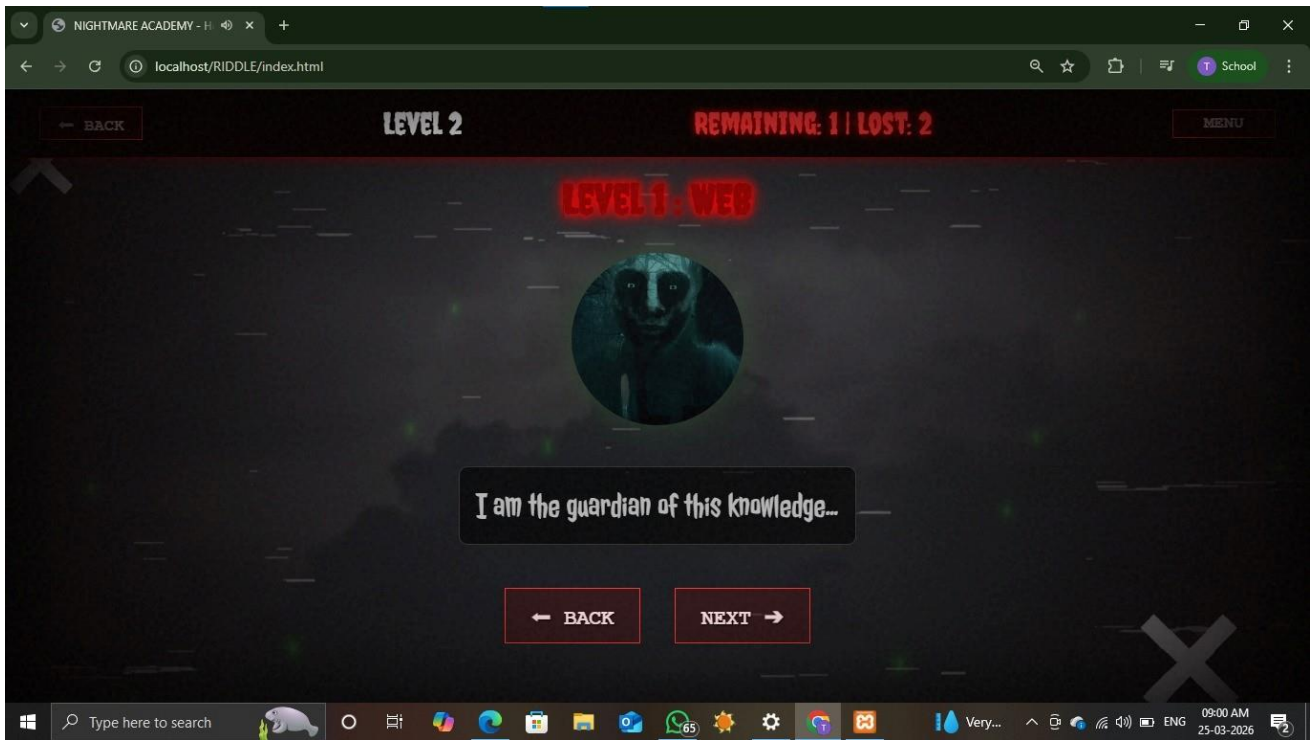


Figure B.5 Navigation Page

REFERENCES

1. I. Sommerville, Software Engineering, 10th ed. Pearson, 2015.
2. R. S. Pressman, Software Engineering: A Practitioner's Approach, 7th ed. McGraw-Hill, 2014.
3. E. Balagurusamy, Programming in Java, 5th ed. McGraw-Hill Education, 2019.
4. J. Duckett, HTML and CSS: Design and Build Websites. Wiley, 2011.
5. J. Duckett, JavaScript and jQuery: Interactive Front-End Development. Wiley, 2014.
6. D. Flanagan, JavaScript: The Definitive Guide, 6th ed. O'Reilly Media, 2011.
7. Mozilla Developer Network, "HTML, CSS, and JavaScript Documentation," [Online]. Available: <https://developer.mozilla.org>
8. W3Schools, "Web Development Tutorials," [Online]. Available: <https://www.w3schools.com>
9. Google Developers, "Web Development Best Practices," [Online]. Available: <https://developers.google.com>
10. Mozilla Developer Network, "Web Audio API Guide," [Online]. Available: <https://developer.mozilla.org>
11. Stack Overflow, "Programming Discussions," [Online]. Available: <https://stackoverflow.com>
12. GeeksforGeeks, "Computer Science Articles," [Online]. Available: <https://www.geeksforgeeks.org>
13. TutorialsPoint, "Web Technology Tutorials," [Online]. Available: <https://www.tutorialspoint.com>
14. freeCodeCamp, "Interactive Web Development," [Online]. Available: <https://www.freecodecamp.org>
15. Khan Academy, "Programming Courses," [Online]. Available: <https://www.khanacademy.org>